Experiment # 6

Implementation of Doubly Linked List

Objective:-

The objective of this session is to understand the various operations on Simple and Circular Doubly linked list in C++.

1. Insertion
2. Deletion
3. Traversal

Software Tools:-

1. DevC++.

Theory:-

A linked list is a collection of components, called nodes. Every node (except the last node) contains the address of the next node. Thus, every node in a linked list has two components: one to store the relevant information (that is, data) and one to store the address, called the next, of the next node in the list. The address of the first node in the list is stored in a separate location, called the head or first. Figure 1 is a pictorial representation of a node.

Each node in the linked list contains:

1. One or more members that represent **data** (e.g. inventory records,customer names, addresses, telephone numbers, etc).

2. 2 **pointers**, that can point to previous and next nodes.

Figure 1: Linked List Node

Linked list: A list of items, called **nodes**, in which the order of the nodes is determined by the addresses, called the **prev** and **next**, stored in each node.

The list in Figure2 is an example of a linked list.

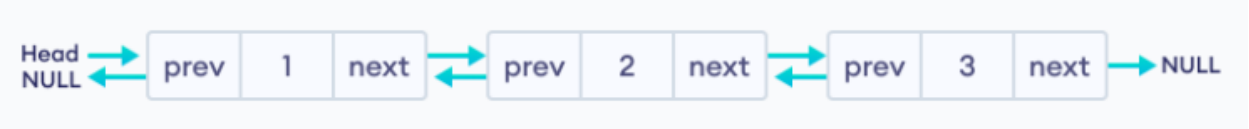

Figure 2: Linked List

The arrow in each node indicates that the address of the node to which it is pointing is stored in that node. The down arrow in the last node indicates that this next field is **NULL**.

Because each node of a linked list has two components, we need to declare each node as a **class** or **struct**. The data type of each node depends on the specific application — that is, what kind of data is being processed. However, the next component of each node is a pointer. The data type of this pointer variable is the node type itself. For the previous linked list, the definition of the node is as follows.
(Suppose that the data type is **int**.)

```
struct node
{
int data;
node* prev, *next;
};
```

The pointer variable declaration is as follows:

```
node* head;
```

Operations on Linked List:

Following operations can be performed on a linked list:-

1. Insertion:

A node can be inserted at

i. The start of the list
ii. End of the list (Appending a node)
iii. Middle of the list

Consider you have to create a sorted linked list.

i. Appending a Node to the Linked List

To **append** a node to a linked list, means adding it to the **end** of the list. The appendNode member function accepts a float argument, num.
The function will -

a) Create a new node.
b) Store data in the new node.
c) If there are no nodes in the list
Make the new node the first node.
Else
Traverse the List to Find the last node.
Add the new node to the end of the list.
End If.

ii. Inserting Node in the Start and Middle of Linked List

Inserting a node in the middle of a list is *more complicated* than*appending* a node.Assume all values in the list are sorted, and you want *all new values*to be *inserted* in their proper position (preserving the order of thelist).We use the same ListNode structure again, with pseudo code. This pseudo code shows the algorithm to find the new node'sproper position in the list, and inserting it there.It is assumed the nodes already in the list are ordered.

- *Create a new node.*
- *Store the data in new node.*
- *If there are no nodes in the list*
  Make the new node the first node.
- *Else*
  Find the first node whose value is greater than or equal the new value, or the end of the list (whichever is first).
- *Insert the new node before the found node, or at the end of the list if no node was found.*
  End If.

2. Deletion of a Node:

This requires 2 steps –

a) Remove the node from the list without breaking the links created by the next pointers.
b) b) Delete the node from memory.

The **deleteNode** member function *searches* for a node with a**particular value** and deletes it from the list. It uses an algorithm similar to the insert Node function.

- *The two node pointers nodePtr and previousPtr are used totraverse the list (as before).*
- *When nodePtr points to the node to be deleted, previousNode->nextis made to point to nodePtr->next.*
- *This removes the node pointed to by nodePtr from the list.*
- *The final step is to free the memory used by the node using thedelete operator.*

3. Traversing a Linked List

The display List is a member function, that traverses the list, displaying the *value* member of each node.

The following *pseudo code* represents the algorithm -

- *Assign list head to node pointer*

- *While node pointer is not NULL*
  Display the value member of the node pointed to by node pointer.
  Assign node pointer to its own next member.
- *End While.*

LAB TASK:

1. Write a C++ program to create Doubly Linked List ADT. Your Linked List must be able to perform the following functionalities:

a) Insertion of an integer (start/middle/end) in sorted linked list.

b) Deletion of a given integer from the above linked list.

c) Display the contents of the above list after deletion.

2. Write a C++ program to create Circular Doubly Linked List ADT. Your Linked List must be able to perform the following functionalities:

a) Insertion of an integer (start/middle/end) in sorted linked list.

b) Deletion of a given integer from the above linked list.

c) Display the contents of the above list after deletion.